# Software Requirements Specification (SRS)

## AI-Based E-Salesman & E-Marketing Executive System

| | |
|---|---|
| **Course** | UE24CS341A – Software Engineering |
| **Project Type** | Jackfruit SE Mini-Project |
| **Team Name** | Code Connect |
| **Document Version** | 2.0 (Scoped for academic mini-project) |
| **Date** | 05 September 2026 |

## Revision History

| Version | Date | Description | Author(s) |
|---|---|---|---|
| 1.0 | 05-09-2026 | Initial full-scope draft | Team Code Connect |
| 2.0 | 05-09-2026 | Scope reduced to a realistically buildable mini-project: chat-first, rule-based logic, simple in-app CRM | Team Code Connect |

## 1. Introduction

### 1.1 Purpose

This document specifies the software requirements for an **AI-Based E-Salesman & E-Marketing Executive System** — a chat-based virtual sales assistant for a small e-commerce store. It answers product questions, recommends products, scores leads based on conversation behavior, and logs everything in a simple built-in CRM. This SRS is scoped to be achievable by a 4-member student team within a single semester mini-project, while still demonstrating all core concepts named in the problem statement.

### 1.2 Scope

The system, referred to as **AI-Salesman**, is a **web-based chat application** with a demo product catalog. It will:

- Let a customer **chat** with an AI agent about products (price, specs, availability, comparisons).
- **Recommend products** using simple rule-based logic (category match, price range, popularity) — not a full machine-learning recommender.
- **Score each lead** using a transparent, rule-based point system (e.g., +points for asking about price, adding to cart, repeat visits) and classify it as Hot / Warm / Cold.
- Log every conversation and lead score into a **built-in CRM module** (a simple database table + admin view) — not a third-party CRM like Salesforce/HubSpot.
- Let the user "add to cart" and see an order summary — a **simulated checkout**, not a real payment gateway.
- Provide a basic **admin page** listing leads sorted by score, so a "sales executive" persona can see who to follow up with.

**Explicitly out of scope for this project** (called out so evaluators know it's a deliberate, justified boundary, not an oversight):

- Voice interaction (speech-to-text/text-to-speech) — listed as a **future enhancement**, not a core deliverable.
- Integration with a real third-party CRM (Salesforce/HubSpot/Zoho) — replaced with an internal CRM module built as part of this project.
- Machine-learning-based recommendation/lead-scoring models — replaced with clearly defined rule-based logic (still satisfies the "lead scoring" and "personalized recommendations" requirement, just implemented simply).
- Real payment processing — replaced with a simulated order-confirmation screen.
- Multi-language support.

### 1.3 Definitions, Acronyms and Abbreviations

| Term | Definition |
|---|---|
| AI-Salesman | The system being built |
| NLP | Natural Language Processing |
| Lead | A customer who has interacted with the chat and shown interest in a product |
| Lead Score | A numeric value computed from rule-based points assigned to specific customer actions |
| Intent | The purpose behind a user's chat message (e.g., "ask price", "compare products") |
| CRM | Customer Relationship Management — here, a simple internal module storing lead/customer data |
| Session | One continuous chat conversation with a customer |
| Admin | The person (sales/marketing role) who views the leads dashboard |

### 1.4 References


- IEEE 830-1998, IEEE Recommended Practice for Software Requirements Specifications
- Jackfruit SE Mini-Project Guidelines 2026, Dept. of CSE (AI/ML), EC Campus

- Chosen NLP/chat API documentation (e.g., an open-source intent classifier, or a hosted LLM API — to be finalized by the team in the design phase)

### 1.5 Overview

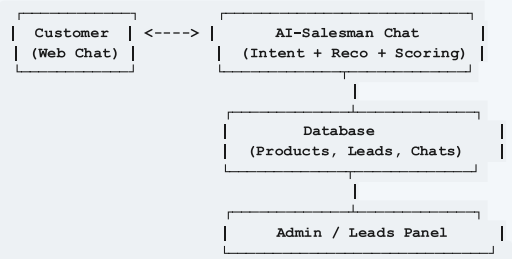
Section 2 describes the product overall, its users, and constraints. Section 3 lists functional requirements by module. Section 4 covers interfaces. Section 5 covers non-functional requirements. Section 6 gives simple system models. Section 7 lists future enhancements (things intentionally left out of this version but good to mention as "next steps").

---

## 2. Overall Description

### 2.1 Product Perspective

AI-Salesman is a standalone web application with three logical parts: a **customer-facing chat widget**, a **backend logic layer** (intent detection, recommendation, lead scoring), and an **admin panel** backed by a database that acts as the internal CRM.




### 2.2 Product Functions (High-Level Summary)

1. Text-based chat interface for customers.
2. Intent recognition for common query types (price, specs, comparison, offers, general).
3. Rule-based personalized product recommendations.
4. Rule-based lead scoring and Hot/Warm/Cold classification.
5. Internal CRM: store customer info, chat logs, lead score, per lead.
6. Simulated cart & checkout summary.
7. Admin dashboard listing leads sorted by score.

### 2.3 User Classes and Characteristics

| User Class | Description | Technical Expertise |
|---|---|---|
| **Customer** | Visits the site, chats with the bot, browses/buys products | None required |
| **Admin (Sales/Marketing role)** | Logs into the admin panel to view leads and chat logs | Basic computer literacy |

### 2.4 Operating Environment

- **Client:** Any modern web browser (Chrome/Firefox/Edge) on desktop or mobile.
- **Server:** Runs on a single backend server/local host during development (e.g., Python/Flask or Node/Express); can be deployed to a free-tier cloud host (Render/Railway/Heroku-equivalent) for demo purposes.
- **Database:** A lightweight database (SQLite/PostgreSQL/MongoDB — to be finalized by the team).

### 2.5 Design and Implementation Constraints

- The project must be completable with free/open-source tools and, at most, a free-tier API key (e.g., a free LLM API tier) — no paid CRM licenses.
- The product catalog will be a small, seeded demo dataset (e.g., 20–50 sample products in one category, like electronics or clothing) — not a live inventory feed.
- Given the academic timeline, the "AI" in query handling can be a hybrid: simple keyword/intent-pattern matching for common cases, with an optional call to a hosted language model API for free-text questions it can't pattern-match.

### 2.6 Assumptions and Dependencies

- The team will manually create/seed the demo product data (CSV/JSON or database seed script).
- If a hosted LLM API is used for open-ended Q&A, the team has access to a free-tier API key.
- The "CRM" is understood by the evaluator to be an internally built module demonstrating CRM *concepts* (lead storage, follow-up flagging), not a commercial CRM integration — this is stated clearly here so it is not seen as a missed requirement.

---

## 3. Specific Requirements (Functional Requirements)

### 3.1 Module: Chat Interface

| ID | Requirement |
|---|---|
| FR-CHAT-01 | The system shall provide a text chat window on the website where customers can type messages. |
| FR-CHAT-02 | The system shall display the bot's replies in the same chat window in real time. |
| FR-CHAT-03 | The system shall remember the current session's conversation (product discussed, preferences mentioned) until the session ends. |
| FR-CHAT-04 | The system shall detect the intent of a message from a defined set: `price_query`, `spec_query`, `compare_query`, `offer_query`, `recommendation_request`, `purchase_intent`, `general/greeting`, `unknown`. |
| FR-CHAT-05 | If intent is `unknown`, the system shall respond with a clarifying question instead of failing silently. |

### 3.2 Module: Product Query Handling

| ID | Requirement |
|---|---|
| FR-PROD-01 | The system shall retrieve product price, specifications, and stock status from the product database and present them in reply to a query. |
| FR-PROD-02 | The system shall support comparing two named products on price and key specs. |
| FR-PROD-03 | The system shall mention any discount/offer flag stored against a product, if present. |

### 3.3 Module: Recommendation Engine (Rule-Based)

| ID | Requirement |
|---|---|
| FR-REC-01 | The system shall recommend products from the same category as the one currently being discussed. |
| FR-REC-02 | The system shall recommend products within ±20% of a budget the customer has mentioned. |
| FR-REC-03 | If no prior context exists, the system shall recommend the top N "popular" products (marked as such in the seed data, e.g., by an `is_featured` flag). |
| FR-REC-04 | The recommendation logic (rules and thresholds) shall be documented in code comments/design doc so it is clearly explainable to an evaluator. |

### 3.4 Module: Lead Scoring (Rule-Based)

| ID | Requirement |
|---|---|
| FR-LEAD-01 | The system shall maintain a numeric lead score per customer session, starting at 0. |
| FR-LEAD-02 | The system shall increment the score by a defined amount for specific actions, e.g.: asking about price (+5), viewing 3+ products (+5), asking for a comparison (+10), mentioning intent to buy (+20), adding to cart (+15). |
| FR-LEAD-03 | The system shall classify the lead as **Cold** (0–14), **Warm** (15–34), or **Hot** (35+) based on the total score. Thresholds are configurable. |
| FR-LEAD-04 | The system shall store the final/current lead score and classification against the customer record in the database. |

### 3.5 Module: Internal CRM

| ID | Requirement |
|---|---|
| FR-CRM-01 | The system shall create a customer/lead record (name or session ID, contact info if provided, timestamp) on first interaction. |
| FR-CRM-02 | The system shall store the full chat transcript against the corresponding lead record. |
| FR-CRM-03 | The system shall update the lead's score and classification after every relevant action in the conversation. |
| FR-CRM-04 | The system shall provide an admin view listing all leads with their score, classification, and last interaction date, sortable by score. |

### 3.6 Module: Purchase Guidance (Simulated)

| ID | Requirement |
|---|---|
| FR-PUR-01 | The system shall allow a customer to add a discussed/recommended product to a simulated cart. |
| FR-PUR-02 | The system shall display an order summary (items, quantities, total price) when the customer chooses to "checkout". |
| FR-PUR-03 | The system shall mark the lead as "converted" in the CRM upon simulated checkout completion (no real payment is processed). |

## 4. External Interface Requirements

### 4.1 User Interfaces

- **Chat widget:** Simple web page with a message list and text input box; product replies can show as plain text or simple cards (image, name, price).
- **Admin panel:** A basic table/list view (leads, score, status), with a page to view a lead's full chat transcript.

### 4.2 Hardware Interfaces

None beyond a standard computer/browser — no microphone/speaker required in this scoped version.

### 4.3 Software Interfaces

| Interface | Purpose |
|---|---|
| Backend Framework (e.g., Flask/Django/Node+Express) | Serves the chat and admin web pages, handles logic |
| Database (e.g., SQLite/PostgreSQL) | Stores products, leads, chat logs |
| (Optional) Hosted LLM API | Handles free-text queries that don't match a known pattern |

### 4.4 Communication Interfaces

- HTTP/HTTPS for all client-server communication.
- Simple REST endpoints between the chat frontend and backend (e.g., `POST /chat`, `GET /admin/leads`).

---

## 5. Non-Functional Requirements

| ID | Requirement |
|---|---|
| NFR-PERF-01 | The system shall respond to a chat message within 2 seconds under normal local/demo conditions. |
| NFR-USE-01 | The chat interface shall be usable by a first-time visitor with no instructions. |
| NFR-REL-01 | If the optional LLM API call fails or times out, the system shall fall back to a default clarifying response rather than crashing. |
| NFR-SEC-01 | Any customer contact info stored (if collected) shall not be exposed on any public-facing page — visible only via the admin panel. |
| NFR-MAIN-01 | The lead-scoring point values and recommendation rules shall be stored in a single configuration file/section, so they can be adjusted without touching core logic. |
| NFR-PORT-01 | The system shall run locally with a documented setup (README with install/run steps) so it can be demonstrated on any team member's or evaluator's machine. |

---

## 6. System Models

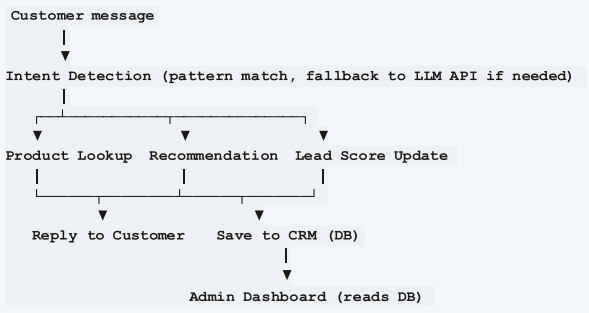
### 6.1 Use Case Diagram (Description)

**Actors:** Customer, Admin

**Use Cases:** 1. Chat with AI-Salesman (Customer) 2. Ask Product Query (Customer) 3. Receive Recommendation (Customer) 4. Add Product to Cart / Checkout (Customer) 5. System Computes Lead Score (System, triggered automatically) 6. View Leads Dashboard (Admin) 7. View Chat Transcript of a Lead (Admin)

*(Draw this as an actual UML diagram — draw.io/Lucidchart/PlantUML — and save under `/docs/diagrams/usecase.png`.)*

### 6.2 Simple Data Flow




### 6.3 Suggested Minimal Data Model

| Table | Key Fields |
|---|---|
| `products` | id, name, category, price, specs, stock_status, is_featured, offer_flag |
| `leads` | id, session_id, name (optional), contact (optional), score, status (Hot/Warm/Cold), created_at |

| | |
|---|---|
| `chat_logs` | id, lead_id, sender (customer/bot), message, timestamp |
| `orders` (simulated) | id, lead_id, product_ids, total_price, status |

## 7. Future Enhancements (Explicitly Out of Current Scope)

Listing these shows the evaluator the team understood the full original problem statement but made a deliberate, justified scoping decision for a one-semester project: - Voice interaction (STT/TTS) for hands-free conversation. - Integration with a real third-party CRM (Salesforce/HubSpot/Zoho) via API. - ML-based recommendation engine (collaborative filtering) and ML-based lead scoring, replacing the current rule-based versions. - Real payment gateway integration. - Multi-language support.

## Appendix A: Glossary

See Section 1.3.

## Appendix B: Team

| Name | Role | GitHub Username |
|---|---|---|
| *ATHARVI* | Repo Admin | atharvi-svg |
| *CHANDANA* | Requirements Analyst | ChandanaRaniS |
| *HARIKA* | Documentation Lead | Harika-S-git |
| *ARUN* | Reviewer | arunkadli |

**Team Name:** Code Connect **Project ID:** *15*